Code Conventions

Software Development Fall 2016

Note:

Since there is no coding standard defined in the C# Language Specification this document holds a collection of programming conventions from guidelines posted on the Microsoft Developer Network.

Contents

References:

[1] Microsoft, "C# coding conventions (C# programming guide)," 2016. [Online]. Available: https://msdn.microsoft.com/en-us/library/ff926074.aspx. Accessed: Oct. 28, 2016.

[2] Microsoft, "Naming guidelines," 2016. [Online]. Available: https://msdn.microsoft.com/en-us/library/ms229002(v=vs.110).aspx. Accessed: Oct. 28, 2016.

[3] Microsoft, "XML documentation comments (C# programming guide)," 2016. [Online]. Available: https://msdn.microsoft.com/en-us/library/b2s063f7.aspx. Accessed: Oct. 28, 2016.

1. Naming

1.1. Capitalization Conventions

- To differentiate words in an identifier, capitalize the first letter of each word in the identifier. Do not use underscores to differentiate words or anywhere else within identifiers.
- There are two appropriate ways to capitalize identifiers depending on their use:

  PascalCasing: capitalize the first letter of each word

  ```
  LessonPlanViewModel
  HomeController
  ```

  camelCasing: capitalize the first character of each word except the first word

  ```
  userRepo
  showArchivedDocs
  ```

- DO use PascalCasing for all public member, type, and namespace names consisting of multiple words.
- DO use camelCasing for parameter names.
- DO NOT capitalize each word in compound words.

1.2. General Naming Conventions

1.2.1. Word Choice

- DO choose easily readable identifier names.
- DO favor readability over brevity.
- DO NOT use Hungarian (type prefix) notation.
- AVOID using identifiers that conflict with keywords of widely used programming languages.

1.2.2. Abbreviations and Acronyms

- DO NOT use abbreviations or contractions as part of identifier names.
- DO NOT use acronyms that are not widely accepted. If they are, only use them when necessary.

1.2.3. Language-Specific Names

- DO use semantically interesting names rather than language-specific keywords for type names

- DO use a common name, such as *value* or *item*, rather than repeating the type name.

1.3. Type Members

1.3.1. Names of Methods

- DO give methods names that are verbs or verb phrases.

1.3.2. Names of Properties

- DO name properties using a noun, noun phrase, or adjective.
- DO NOT have properties that match the name of "Get" methods, this typically indicates that the property should be a method.
- DO name collection properties with a plural phrase describing the items in the collection instead of using a singular phrase followed by "List" or "Collection."
- DO name Boolean properties with an affirmative phrase. You can also prefix Boolean properties with "Is," "Can," or "Has."
- CONSIDER giving a property the same name as its type.

1.3.3. Names of Events

- DO name events with a verb or a verb phrase.
- DO give events names with a concept of before and after using the present and past tenses.
- DO NOT use "Before" or "After" prefixes or postfixes to indicate pre-events and post-events.
- DO name event handlers with the "EventHandler" suffix.
- DO use two parameters named *sender* and *e* in event handlers.
- DO name event argument classes with the "EventArgs" suffix.

1.3.4. Names of Fields

- These guidelines apply to static public and protected fields.
- DO use PascalCasing in field names.
- DO name fields using a noun, noun phrase, or adjective.
- DO NOT use a prefix for field names.

1.4. Parameters

- DO use camelCasing in parameter names.
- DO use descriptive parameter names.
- DO use names based on a parameter's meaning rather than the parameter's type.

2. Layout Conventions

- Use the default code editor settings: smart indenting, four-character indents, tabs saved as spaces.
- Write only one statement per line.
- Write only one declaration per line.
- Indent continuation lines by one tab stop (four spaces).
- Add at least one blank line between method definitions and property definitions.
- Uses parenthesis to make clauses in an expression apparent.
- Newline all opening and closing curly braces unless they enclose an auto-implemented property's `get` and `set` accessors.

```
public Term()
{
    Courses = new HashSet<Course>();
    Holidays = new HashSet<Holiday>();
}
public string Name { get; set; }
```

3. Commenting Conventions

- Place comments on a separate line, not at the end of a line of code.
- Begin comments with an uppercase letter and end them with a period.
- Insert a space between the comment delimiter and its text.
- Do not create formatted blocks of asterisks around comments.
- Avoid inline comments, they can be a sign of code that is confusing.

- Documentation can be created by including XML elements in special comment fields (indicated by triple slashes) in source code directly before the code block which the comments refer.

```
/// <summary>
///  This class performs an important function.
/// </summary>
public class MyClass{}
```

- All public methods should have documentation in the format listed above.

4. Language Guidelines

4.1. Implicitly Typed Local Variables

- Use implicit typing for local variables when the type of the variable is obvious from the right side of the assignment or when the precise type is not important.

```
var var1 = "This is clearly a string.";
var var2 = 27;
```

- Do not use `var` when the type is not apparent from the right side of the assignment.
- Do not rely on the variable name to specify the type of the variable, it might not be correct.
- Avoid the use of `var` in place of `dynamic`.

4.2. New Operator

- Use the concise form of object instantiation with implicit typing.
- Use object initializers to simplify object creation.

4.3. Static Members

- Call static members by using the class name: *ClassName.StaticMember*.
- Do not qualify a static member defined in a base class with the name of a derived class.

4.4. LINQ Queries

- Use meaningful names for query variables:

```
var seattleCustomers = from cust in customers
                       where cust.City == "Seattle”
                       select cust.Name;
```

- Use aliases to make sure that property names of anonymous types are correctly capitalized using PascalCasing.
- Rename properties when the property names in the result would be ambiguous.

```
var localDistributors2 = from cust in customers
                         join dist in distributors
                         on cust.City equals dist.City
                         select new
                         {
                             CustomerName = cust.Name,
                             DistributorID = dist.ID
                         };
```

- Use implicit typing in the declaration of query variables and range variables.
- Align query clauses under the `from` clause.
- Use `where` clauses before other query clauses to ensure that later query clauses operate on the reduced, filtered set of data.
- Use multiple `from` clauses instead of a `join` clause to access inner collections.